Computer Graphics: Lecture 4

Your first Triangles

Welcome Back!

Last time we developed our first 3D graphics application!

But it wasn't very 3D…or 2D…or any D.

That's because we were just clearing the screen.

Obviously that's not very interesting. We want to draw things. And what is the simplest thing to draw? Triangles!

Quick review

Before we start drawing things, let's review what we know:

- There's something called the 3D graphics pipeline. The triangles go through that pipeline and are transformed into pixels.
- We don't just call a "drawTriangles" function and walk away. Instead, we build a command buffer that tells the GPU to draw the triangles. We compile that buffer, and we send it to the GPU.
- The result is copied into the canvas in our webpage (or the background of a window if on desktop).

So, how do we do that?

Commands to draw triangles

Previously, we cleared our screen by defining a render pass with the correct clear color and attachment operations:

```
const encoder = device.createCommandEncoder();
const pass = encoder.beginRenderPass({
    colorAttachments: [{
        loadOp: 'clear',
        storeOp: 'store',
        view: context.getCurrentTexture(),
        clearValue: {r: .7, g: .8, b: .9, a: 1.0},
    }]
});
pass.setViewport(0, 0, context.canvas.width, context.canvas.height, 0, 1);
pass.end();
device.queue.submit([encoder.finish()]);
```

Drawing

There are (at least) two more commands we have to add to draw.

The first is that we have to tell it which pipeline to use. We haven't defined a pipeline yet, so this is a placeholder.

Then, we have to tell it to `draw` some points. We tell it to draw 3 points, which it will stitch together into a triangle.

```
pass.setViewport(...); // more on viewports later
pass.setPipeline(pipeline); // we must create this "pipeline"
pass.draw(3); // draw 3 points.
pass.end();
```

The importance of a pipeline

Drawing requires a pipeline. You can't draw without a pipeline because it wouldn't know where to get the data.

"Okay, you want me to draw a triangle, but where? With what points? What color? What shaders do you want me to run?"

The pipeline will describe where to fetch the raw data, what shaders to run on it, where to put the results, and some other interesting things.

We're working backwards. I find it's easier to remember if we start with what we want (the draw call) and work backwards to build the objects we need. Let's do the "next" step and build a pipeline.

Building a pipeline

We build a pipeline by calling `device.createRenderPipeline(...)`[1]

```
const pipeline = device.createRenderPipeline({
    layout: 'auto',
    vertex: {
        module: shaderMod, // we haven't defined shaderMod yet
    },
    fragment: {
        module: shaderMod,
        targets: [
            {format: context.getCurrentTexture().format,}
        ],
    },
});
```

[1] There is also a `createComputePipeline` for running raw code on the GPU. The compute pipeline is simpler, but we need all the special hidden features WebGPU supports for triangle rendering.

Describing the pipeline

Let's talk about what went into that pipeline.

First, we specified its `layout` as `'auto'`. The layout describes the order and location of external data that is available to the shaders. We don't need any yet, so we set this to be automatically determined.

Then, we have two descriptors for `vertex` and `fragment`. These describe the vertex and fragment (pixel) shaders. We'll talk about what these shaders do in a bit, but know that they appear in practically every 3D pipeline. We will define the `shaderMod` variable next.

`targets` gives information about the data the fragment shader is writing to. Again, we'll cover this soon.

Building a shader module

Think back to when you learned C. You did not run the C code directly in an interpreter: instead you compiled it into an object file. Object files are binary code that can be linked together into an executable or library.

In the same way, we compile shader code into object files (the "linking" isn't something we have to worry about).

So, we give WebGPU our shader source code and it will compile it into a binary that can be further transformed, loaded, or linked by the GPU.[1]

[1]The binary format is called SPIR-V (pronounced "spear-vee" and standing for "standard portable intermediate representation…vee" The V doesn't officially stand for anything, but some people think it stands for "Vulkan", because that's the API it was intended for.

Building a shader module

Building a shader module for the pipeline is easy:

```
const shaderMod = device.createShaderModule({
    code: stringThatHasTheShaderCode,
    label: "a description of the shader" // optional
});
```

There is one optional field where you can provide compilation hints, but we won't bother to use it.

And now we can't avoid it anymore. We have to write the shader code. That `stringThatHasTheShaderCode` has to have something in it.

But first...

Questions?

WGSL

Most modern 3D graphics workflows let you program shaders in whatever language you want, and compile them into SPIR-V.

Unfortunately, claiming reasons involving a legal dispute, Apple was unable to agree to a format controlled by an outside party (source).

Instead, the committee decided on a programming language called WGSL (WebGPU shading language, originally named “tint”, pronounced “wigg-sl”)

WGSL syntax

This language is somewhat controversial. It's rather verbose, and its syntax appears to be based on Rust and Typescript rather than C (like the old OpenGL shading language was).

However, since we are using Typescript, that doesn't seem like a bad thing. Personally, I don't find the syntax too horrible (but I am familiar with Rust, which it resembles most closely).

Let's write a simple shader for drawing our triangles. I'll show the shader first, and then explain it…

Basic triangle shader

```
// we store it in a string. /*wgsl*/ is for syntax highlighting with plugins.
const vertsInShaderCode = /*wgsl*/` //<- this is a backtick string: ` not '
const vert_positions: array<vec4f, 3> = array(
    vec4f(-.75, -.75, 0, 1),
    vec4f( .75, -.75, 0, 1),
    vec4f(   0,  .75, 0, 1),
);

@vertex fn vs(@builtin(vertex_index) index:u32) -> @builtin(position) vec4f {
    return vert_positions[index];
}

@fragment fn fs() -> @location(0) vec4f {
    return vec4f(0.4, 0.8, 0.3, 1.0);
}
` // <- closing backtick
```

Basic triangle shader

Okay, let's explain what's going on

This shader is defined inside our Typescript source code. It's stored inside of a variable.

We could store it in an external file and load it, but this would require using some web features I don't want to cover yet (`fetch` and `async`)

We use a backtick string, officially called a "template literal". In Typescript, you can define strings with `""` or `''` like python, but you can also define them with ``` ``

We do this because template strings can stretch across lines.

Basic triangle shader (2)

The next thing to discuss is the `/*wgsl*/` comment at the top.

This isn't required, it's just a comment, so it gets stripped out and ignored by the compiler/browser.

It's there because I'm using a plugin that allows WGSL source code to be syntax highlighted inside of Typescript or Javascript files.

I recommend using it too, it's called "WGSL Literal" in the VSCode marketplace. (there's also a WGSL one for `.wgsl` files for when you get to a point where you can load an external file).

Basic triangle shader (3)

Let's talk about the body of the shader.

```
const vert_positions: array<vec4f, 3> = array(
    vec4f(-.75, -.75, 0, 1),
    vec4f( .75, -.75, 0, 1),
    vec4f(   0,  .75, 0, 1),
);
```

There's a lot going on here:

- `const` is for defining constants. It's like `const` in Typescript, except you *cannot* modify its fields. It's a true compile-time constant.
- Types are specified like in Typescript or Rust. A colon followed by the type name. In this case `array<...>` is defining a type.

Basic triangle shader (4)

```
const vert_positions: array<vec4f, 3> = array(
    vec4f(-.75, -.75, 0, 1),
    vec4f( .75, -.75, 0, 1),
    vec4f(   0,  .75, 0, 1),
);
```

- When `array<...>` defines a type, we use angle brackets to describe what it's an array of and how many elements it has.
- `vec4f` is the element type. This means a vector of 4 elements, each of which are floats. It's a shortcut for writing `vec4<f32>`. `f32` is how `float` is written in this language (like in Rust).
- Why are there 4 dimensional vectors? We'll talk about that in a bit.
- The `3` is the length of the array.

Basic triangle shader (5)

```
const vert_positions: array<vec4f, 3> = array(
    vec4f(-.75, -.75, 0, 1),
    vec4f( .75, -.75, 0, 1),
    vec4f(   0,  .75, 0, 1),
);
```

- so `array` is a type, but it's also a constructor. This is a common pattern in WGSL. We use `array(...)` to define the array.
- Inside the array are 4 values: x, y, z, and w. What coordinate system are we using? We'll talk more about it later in this lecture, but basically, $x = -1, y = -1$ is the bottom left of the screen and $x = 1, y = 1$ is the top right. Don't worry about z and w for now.
- We terminate the definition with a `;`, just like in C.

Basic triangle shader (6)

Okay, we've seen "what" the array is, but not "why" the array...

This array describes the 3 vertex positions of our triangle.

This information is used by the vertex shader:

```
@vertex fn vs(@builtin(vertex_index) index: u32) ->
  @builtin(position) vec4f
{
    return vert_positions[index];
}
```

This is a single function, which is the entrypoint (main) for the vertex shader. The `@vertex` attribute tells us that.

Basic triangle shader (7)

```
@vertex fn vs(@builtin(vertex_index) index: u32) ->
  @builtin(position) vec4f
{
    return vert_positions[index];
}
```

- The `fn` keyword defines a function.
- `vs` is the name of the function. It can be anything we want: I name it `vs` for “vertex shader”.
- We can specify the name in the pipeline instead and leave `@vertex` off if we want, but I never find that more convenient than `@vertex`.
- The vertex shader takes one piece of data for every vertex: the *index* of that vertex (i.e., which vertex it is).

Basic triangle shader (8)

When we draw a 3D object (called a mesh), it will contain usually hundreds or thousands of vertices.

Each vertex has a number describing it, called its index.

Indices usually start at 0 and go upwards as far as we need.

Every time we want to draw a vertex, the vertex shader is called on it, like a function. It can receive whatever data about that vertex we want. Each piece of data is a parameter of the vertex shader function.

Basic triangle shader (9)

```
@vertex fn vs(@builtin(vertex_index) index: u32) ->
  @builtin(position) vec4f
{
    return vert_positions[index];
}
```

WGLS must know where every variable comes from. Saying `@builtin(vertex_index)` means that the parameter will come from the built-in index value of the vertex.

We are basically renaming this special variable to `index`, and saying that we want it to be a `u32` (an unsigned 32-bit integer). We could have made it a `u16` since we don't have very many points.

Basic triangle shader (10)

If we had left off the `@builtin(vertex_index)` from in front of the parameter, we would have gotten an error, because the system would not know where to get the parameter from.

Similarly, the `->` is a return value. This return value is also going to a built-in location: `position`.

Every vertex is required to return something with the `@builtin(position)` set. The position is important because it is clipped and transformed further after it is returned by the vertex shader, so the system has to know which return value is the position (there can be more than one returned vector).

With me so far?

In between the vertex and fragment shader

The purpose of the vertex shader is to bring a vertex from **model space** (i.e., the coordinates used in the 3D editor, like Blender) into **clip space** (the [−1, 1] coordinate system we mentioned briefly).

In our vertex shader, we predefined the coordinates of the triangle in an array. The shader uses the index to look up the specific coordinates for a given vertex number, and it returns that value.

After the vertex shader runs, the underlying 3D hardware links the vertices into triangles, figures out which pixels they cover, and runs the fragment shader on each of those pixels. So, let's talk about the fragment shader...

Basic triangle shader (11)

```
@fragment fn fs() -> @location(0) vec4f {
    return vec4f(0.4, 0.8, 0.3, 1.0);
}
```

- We know this is the fragment shader because of the `@fragment`
- Fragment shaders normally take data about the triangle they are on the surface of, but this fragment shader doesn't take any parameters.
- The fragment shader also returns a `vec4f`, but it's interpreted as a color. Instead of `xyzw` coordinates, it has `rgba` coordinates, for "red", "green", "blue", and "alpha"[1].

[1]Technically, it's the same data type. You can refer to a vector with .xyz or with .rgb, and they are interchangable. `r` just means `x`, `g` means `y`, etc. We'll talk about alpha when we get to color blending

Basic triangle shader (12)

```
@fragment fn fs() -> @location(0) vec4f {
    return vec4f(0.4, 0.8, 0.3, 1.0);
}
```

The fragment shader is returning a color at `@location(0)`. Fragment shaders can return more than one thing. In our case, 0 means the 0th render attachment, which is the one we're drawing to our canvas.

The actual shader is very simple: it returns a constant color value (a greenish color) for every pixel. So the triangle the GPU draws will have a single constant color. Not very interesting, but we'll do more with colors, textures, and shading, later.

A Summary

- We defined our shader code in WGSL as a string
- We compiled it with `device.createShaderModule(...)`
- We created a pipeline with `device.createPipeline(...)`
- The pipeline had fields for layout (auto), a vertex shader, and a fragment shader. These fields told it to use the shader module we compiled. They also described the output format in the fragment shader.
- We created a command encoder. We defined a render pass with the correct attachment information (clear color, operations, and target).
- We defined the pass: its viewport, the pipeline, and we said `draw`
- We ended the pass, finished the command, and sent it to the GPU

Finishing up

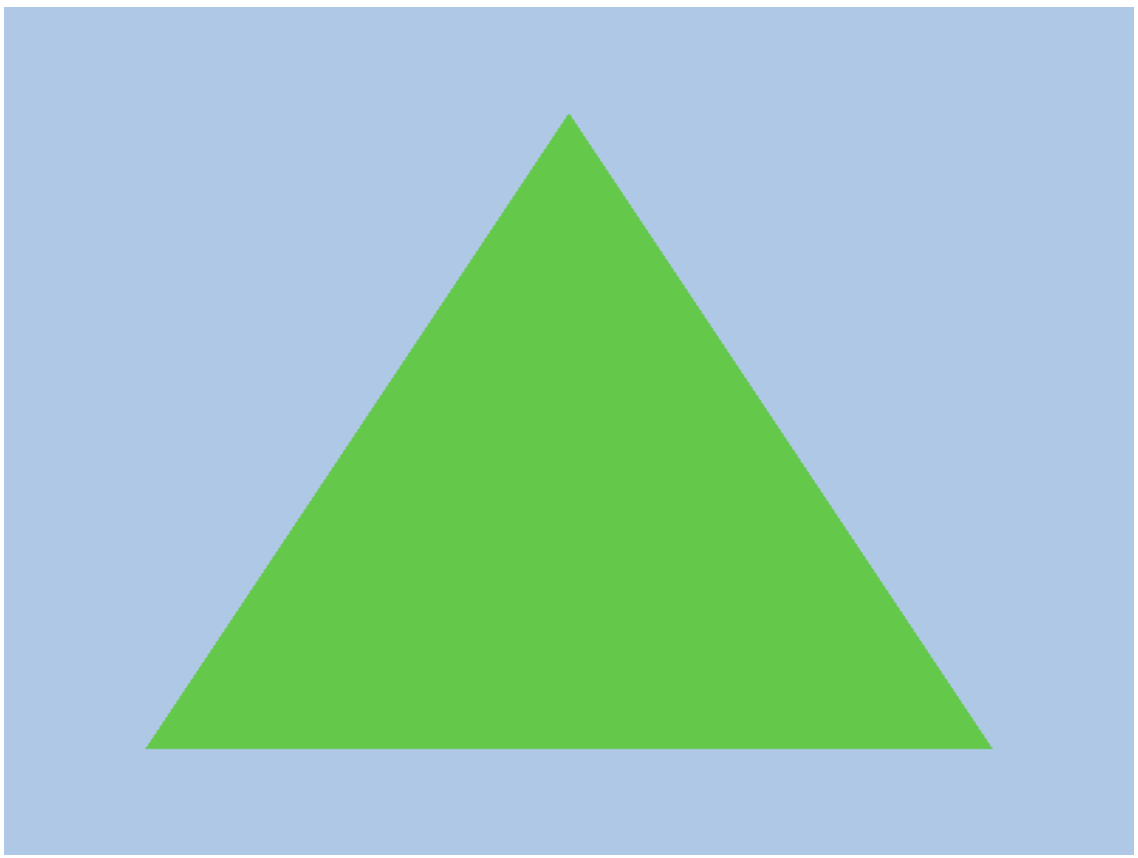
Go ahead and follow along as best you can from the slides.

I recommend trying to do as much from memory as you can to help the different steps stick in your memory.

However, you can also look at `sample02` if you get stuck.

Try your best to remember the steps. Quiz yourself and see if you can do it from scratch. This will be worth it when we get to more complicated techniques.

The triangle



Questions?

Let's learn the pipeline properly now

There are two things that are missing from our understanding so far:

1. The pipeline has a lot of implicit stages. How are those coordinates interpreted? How does the system know how to fill in the triangle?
2. We do not want to hardcode the coordinates of the triangle in the shader. One shader is often useful for many objects. The memory available inside the shader is quite limited. We want to store the triangle in a buffer instead.

Let's start by better understanding the pipeline

Understanding the sequence of operations

We submitted a command buffer to the GPU. One command was `draw`.

The GPU will invoke the vertex shader once for every drawn vertex. Our command was `draw(3)`, so the vertex shader will be called 3 times.

Each of these calls is totally independent, so the GPU is free to run each one on its own thread.

Most GPUs have hundreds or even thousands of independent computation units (e.g., CUDA cores or stream processors), so even if we draw 50 vertices there's a decent chance their vertex shaders would all run at the same time.

The pipeline: vertex shader input

Each vertex has a unique number, and that number is passed to its shader as an argument. We could have set up our pipeline to pass more data besides that number (and we will do so momentarilly).

The vertex shader has one purpose: to return a *new* vertex.

Why? Because the raw vertices usually came out of a 3D modelling program. They aren't facing the right way with respect to the camera. They aren't at the correct position in the world. They might be T-posed instead of animated.

The pipeline: vertex shader output

So the vertex shader runs and applies all those transformations.

It then returns a vertex that is in **clip coordinates**.

The 3D system will draw any vertex that is inside the clipping volume, where x and y are both in the inclusive range [-w, w], and z is in the inclusive range [0, w].

We will explain w later, but for now, w will be 1, so this volume is usually [−1, 1] by [−1, 1] by [0, 1].

Primitive assembly

After running the vertex shader independently on each vertex, the next stage in the pipeline is **primitive assembly**

This means that 3 vertices are linked together into triangles.

This happens based on the order of vertices. By default, every group of three vertices is linked into one triangle. (there are other topologies)

We drew 3 vertices, so we produced one triangle. If we had drawn 6, we would have produced 2.

You can't have part of a triangle, so if we had drawn 2, nothing would be shown, and drawing 5 would result in 3.

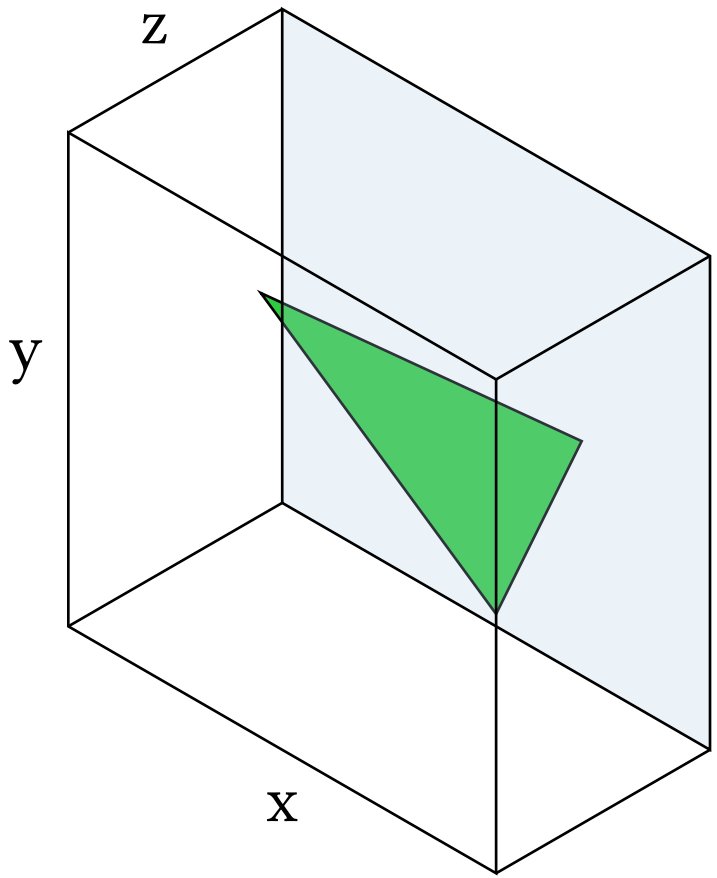
Clipping

The next stage is clipping, which means ensuring that every visible triangle is entirely in view. The purpose of this is to make it so that we don't have to worry about pixels going off screen (and having to check every single pixel would be slow).

- If an the triangle is entirely out of view, it is removed from further processing. This can dramatically speed things up.
- If the entire triangle is in view, it is not clipped at all.
- If the triangle is partially out of view, it is modified or broken up into multiple in-view triangles.

But what is *in view*? Answer: whatever is in the clipping volume.

The clipping volume

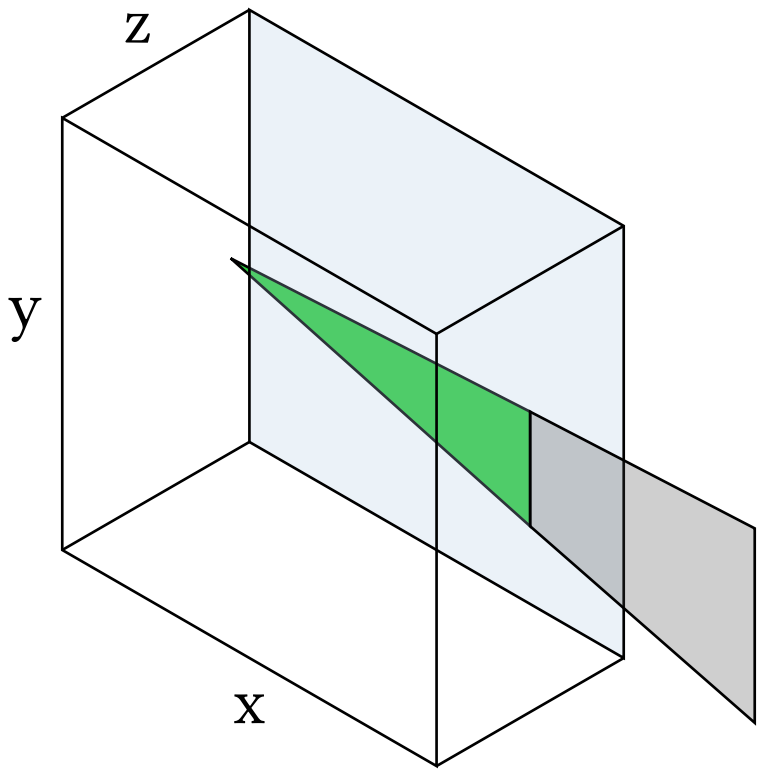


This is what the clipping volume looks like.

The GPU only draws whatever is inside here.

The vertices inside the volume are *after* running the vertex shader on the input. The vertex shader is supposed to produce vertices relative to this volume.

The pipeline: Clipping



If the vertex shader returns a triangle with 1 or more points outside, those points get clipped (shown with 2 points outside).

In some cases, more triangles need to be created. The actual algorithm is up to the vendor.[1]

[1] I won't go into detail how this happens because the GPU does it entirely for us and nearly transparently. Basically, it's finding the edges that intersect the 6 planes of the clipping volume, cutting them to fit, and making new sets of triangles out of the new vertices.

The w-divide (aka “perspective divide”)

What’s the deal with that w-coordinate?

In 3D graphics, we use a 4D coordinate system called homogenous coordinates. The reasons for this will be learned later in the course.

For now, think of “w” as being a “distance ratio”. If w = 4 for a point, it means that it is “four times as far away” as for a point with w = 1.

Try experimenting with different w values for your triangle. If you set one vertex to have a w of 2, it will “push it back”.

To achieve this effect, the GPU *divides* x, y, and z by w.

The w-divide (2)

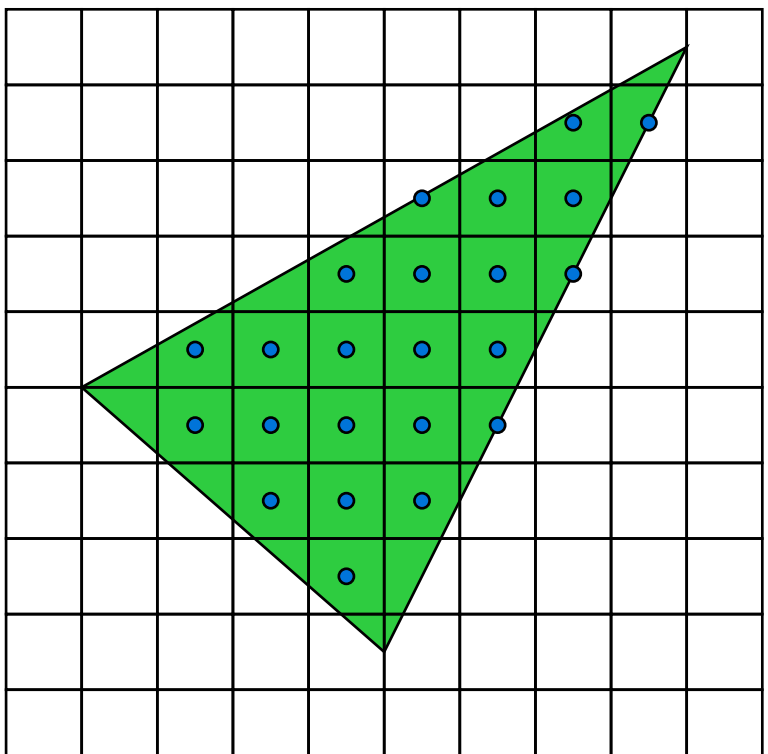
The result of the w divide is that for all visible points, x and y will be in the range $[-1, 1]$, z will be in the range $[0, 1]$, and w won't be needed anymore.

This means, if you set w to 2 for all the points of a triangle, the result will be half the x and y, which will have the effect of making it twice as small.

z also gets shrunk, but you won't see the result on screen. You may have noticed that as long as z is in the range $[0, w]$, it seems to have no effect on the triangle. The actual "perspective effect" does not happen automatically.

The viewport transformation

Fragment Shaders



todo: Explanation